Universidad De las Fuerzas Armadas - Espe

Nombre: Carrera Romero Anthony Sebastian

Fecha: 11/10/2026

NRC: 36460 Carrera: Carrera: Electrónica y automatización

Profesora: Ing. Jenny Robalino

Unidad 1°: Ejercicios de programación y algoritmia

Problema 1.1. Media de dos valores

Desarrolla un programa que ingrese dos números por teclado e imprima su media.

1. Analisis del problema

- Entrada: dos números (num1, num2) ingresados por teclado.
- Proceso: Sumar ambos números y dividir el resultado entre 2.
- Salida: el valor de la media calculada.
- Restricción: los valores de entrada deben ser enteros o reales

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	num1	variable	Entero
Dado de entrada	num2	variable	Entero
Dado de salida	media	variable	Entero

3. Seudocódigo

Algoritmo MediaValores

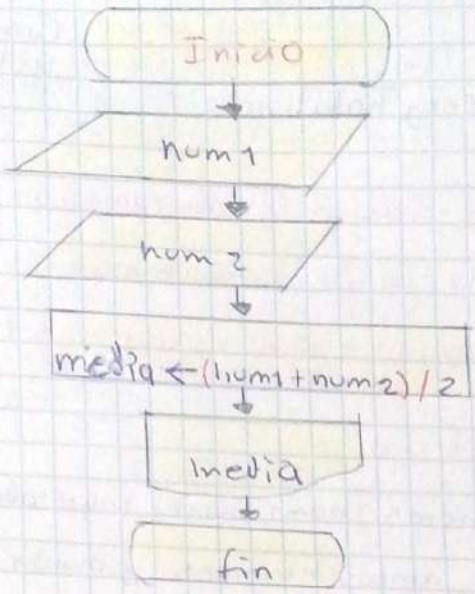
Definir num1, num2, media como Real
Escribir "Ingresar el primer número:"
Leer num1
Escribir "Ingresar el segundo número:"
Leer num2

medio ← (num1 + num2) / 2

Escribir " la media de los dos números es: ", media

FinAlgoritmo

4. Diagrama de flujo



5. Prueba de escritorio

Nº	num 1	num 2	Proceso media ← (num1 + num2) / 2	Salida (media)
1	10	20	(10 + 20) / 2	15
2	7	8	(7 + 8) / 2	7.5
3	-4	10	(-4 + 10) / 2	3

Problema 1.2 Valor absoluto de x al cubo

Desarrollar un programa que lea un número por teclado y y escriba por pantalla $|x|^3$.

1. Análisis del problema.

- Entrada: un número real o entero.
- Proceso: calcular el valor absoluto de x y elevarlo al cubo.
- Salida: el valor resultante.

•Restricción: el valor de entrada debe ser numérico

2. Tabla de objetos

Objetos	Nombre	Valor	Tipo
Dato de entrada	x	variable	Real / Entero
Dato de salida	resultado	variable	Real

3. Seudocódigo

```
Algoritmo ValorAbsolutoCubo
    Definir x, resultado Como Real
    Escribir "Ingrese un número: "
    Leer x
    resultado ← abs(x)^3
    Escribir "El valor absoluto al cubo es: ", resultado
FinAlgoritmo
```

4. Diagrama de flujo



5. Diagrama de flujo

N°	x	Proceso (resultado ← abs(x)^3)	Salida (resultado)
1	2	abs(2)^3	8
2	-3	abs(-3)^3	27

3	-1,5	abs(-1,5)^3	3.37

Problema 1.3. Intervalo

Desarrolla un programa que lea dos números por teclado y determine si el primero es divisible por el segundo. Se mostrará por pantalla el resultado.

1. Análisis del problema

- Entrada: dos números enteros (num1, num2).
- Proceso: dividir el primer número entre el segundo usando el operador módulo (resultado ← num1 % num2).
- Salida: mostrar un mensaje si el primer número es divisible por el segundo o no.
- Restricción: el segundo número (num2) no puede ser cero.

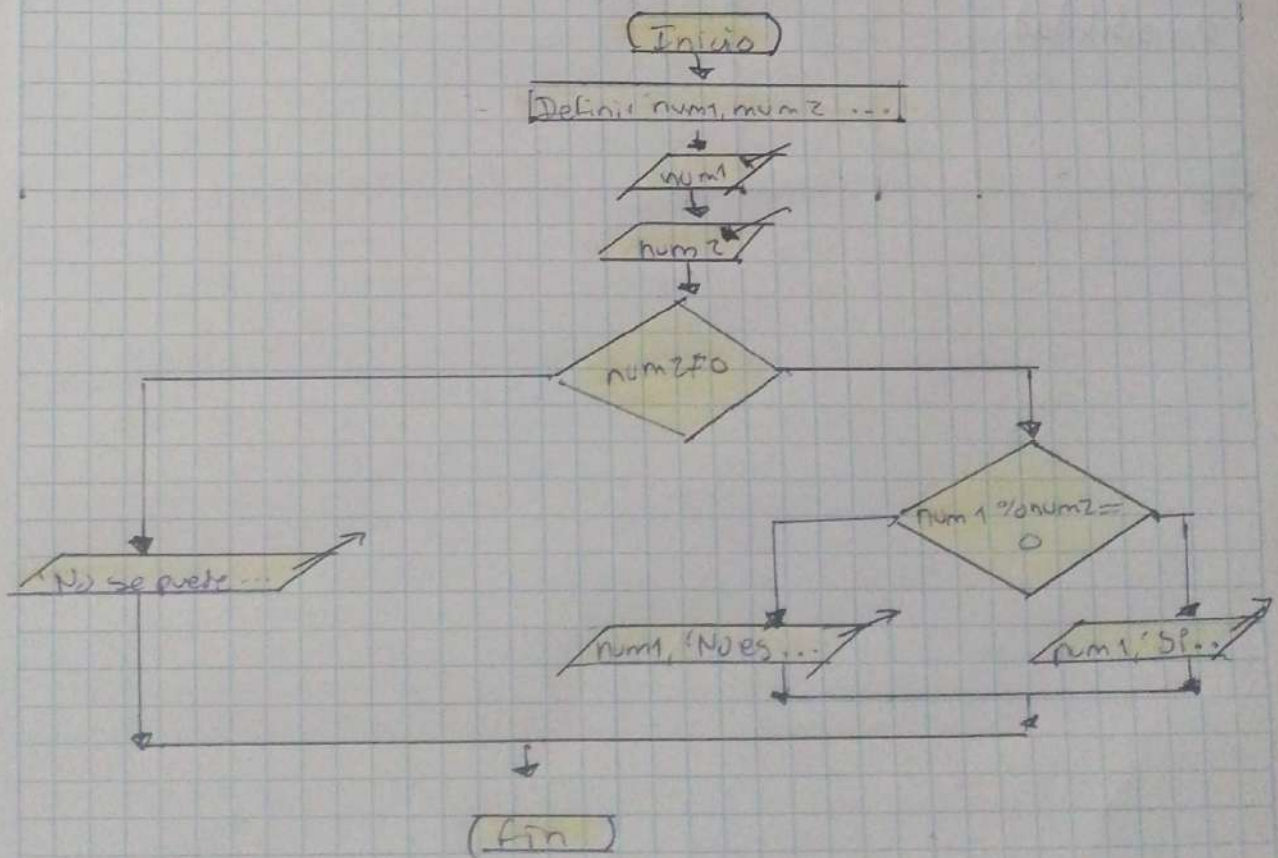
2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	num1	variable	Entero
Dato de entrada	num2	variable	Entero
Proceso	residuo	variable	Entero
Restricción	num2 ≠ 0	variable	lógico

3. Seudocodigo

```
Algoritmo NumerosDivisibles
    Definir "Ingrese el primer número: "
    Leer num1
    Escribir "Ingrese el segundo número: "
    Leer num2
    Si num2 ≠ 0 Entonces
        Si num1 % num2 == 0 Entonces
            Escribir num1, " Si es divisible por ", num2
        SiNo
            Escribir num1, " No es divisible por ", num2
        FinSi
    SiNo
        Escribir " No se puede dividir por cero. "
    FinSi
FinAlgoritmo
```

4. Algoritmo



5. Prueba de escritorio

Operación	num1	num2	Salida en Pantalla
Leer num1, Leer num2	10	2	Ingrese el primer numero (num1): 10 Ingrese el segundo número(num2): 2
Evaluar condición y+0 (verdadero)	10	2	-
Evaluar condición 10%2==0 (0==0, verdadero)	10	2	-
Ejecutar instrucción de Divisibilidad.	10	2	10 si es divisible por 2

Problema 1.4. Intervalo

Desarrolla un programa que lea un número por teclado y determine si pertenece al intervalo [0, 10], indicando el resultado por pantalla.

1. Análisis del problema

- Entrada: un número entero o real (x)
- Proceso: comprobar si ($x \geq 0$ y $x \leq 10$)
- Salida: un mensaje indicando si pertenece o no al intervalo
- Restricción: el número debe encontrarse entre [0, 10]

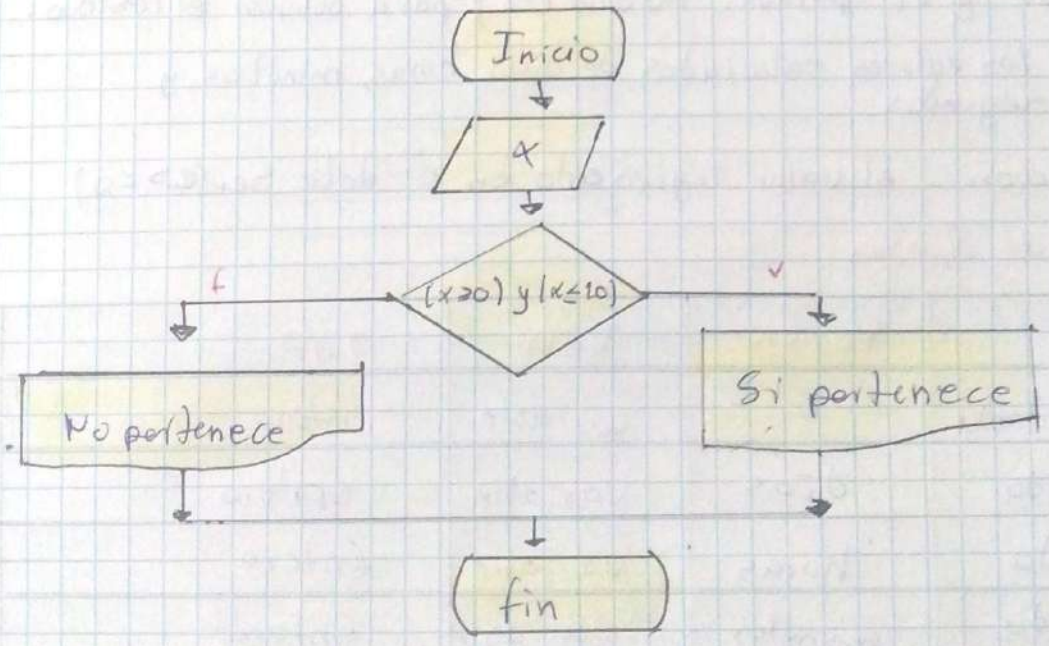
2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	x	variable	Entero / Real
Dato de salida	mensaje	" "	Texto / Caracter
Restricción	0 <= x <= 10	[0, 10]	Lógica

3. Seudocódigo

```
Algoritmo PerteneceIntervalo
    Definir x como Real
    Escribir "Ingrese un número:"
    Leer x
    Si (x>=0) Y (x<=10) Entonces
        Escribir "El número ", x, "Sí pertenece al intervalo [0,10]"
    Sino
        Escribir "El número ", x, "No pertenece al intervalo"
    FinSi
FinAlgoritmo
```

4. Diagrama de Flujo



5. Prueba de escritorio

Nº	x	Condición (x >= 0 y x <= 10)	Salida
1	5	(5 >= 0) y (5 <= 10) verdadero	"Sí pertenece al intervalo"
2	12	(12 >= 0) y (12 <= 10) falso	"No pertenece al intervalo"
3	0	(0 >= 0) y (0 <= 10) verdadero	"Sí pertenece al intervalo"

Problema 1.5. Conversión de unidades de tiempo

Desarrolla un programa que lea por teclado un valor entero c, interpretándolo como el número de segundos que dura un evento, y calcule y muestre por pantalla cuántos días, horas, minutos y segundos representa.

1. Análisis del problema

- Entrada: Un valor c que representa el número total de segundos a convertir.
- Proceso:
  - Un día tiene (24 · 60 · 60) = 86400 segundos
  - Una hora tiene 60 · 60 = 3600 segundos
  - Un minuto tiene 60 segundos.
  - Se utiliza la división entera (trunc) para aislar la unidad mayor y el operador módulo (%) para obtener el residuo.
- Salida: los valores calculados de días, horas, minutos y segundos.
- Restricción: el valor ingresado en c debe ser (c >= 0)

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	c	variable	entero
Dato de salida	días	variable	Entero
Dato de salida	horas	variable	Entero
Dato de salida	minutos	variable	Entero
Dato de salida	segundos1	variable	Entero
Variable de proceso	resto	variable	Entero

3. Seudocódigo

```
Algoritmo ConversionUnidadesTiempo

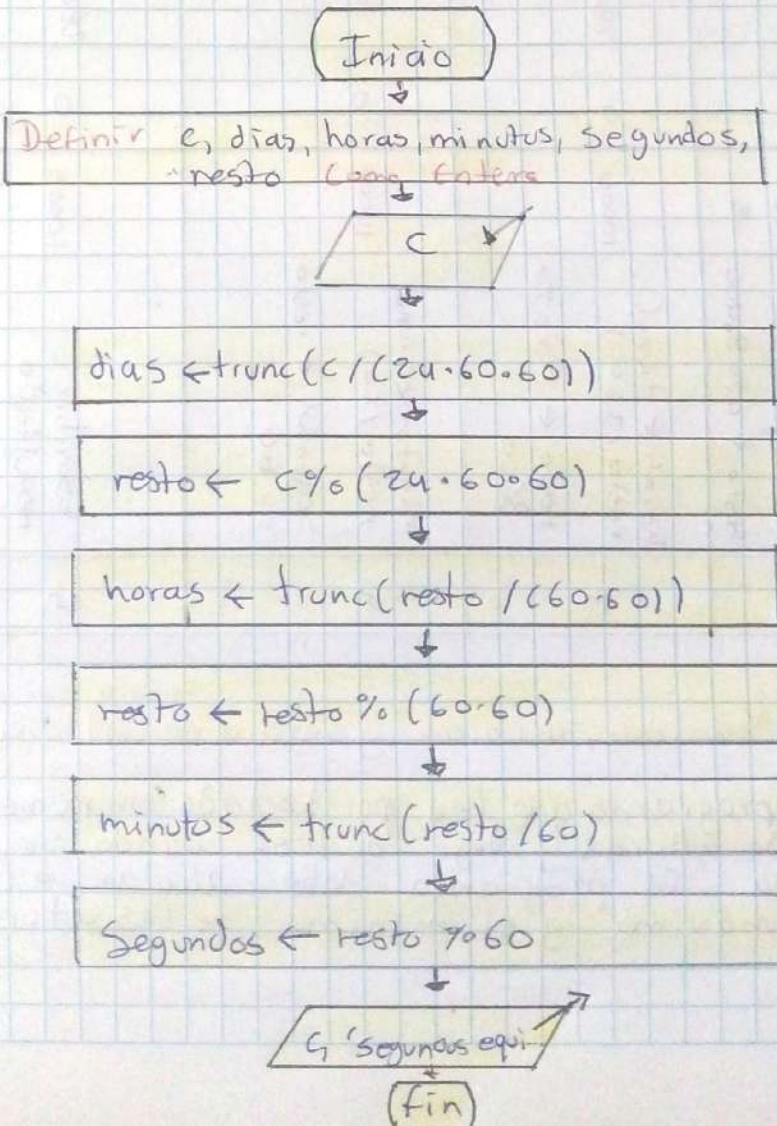
    Definir c, dias, horas, minutos, segundos, resto Como Entero
    Escribir "Ingrese el número de segundos (c): "
    Leer c

    dias ← trunc ( c / ( 24 * 60 * 60 ) )
    resto ← c % ( 24 * 60 * 60 )
    horas ← trunc ( resto / ( 60 * 60 ) )
    resto ← resto % ( 60 * 60 )
    minutos ← trunc ( resto / 60 )
    segundos ← resto % 60

    Escribir "Conversión de unidades: "
    Escribir "Días: ", dias
    Escribir "Horas: ", horas
    Escribir "Minutos: ", minutos
    Escribir "Segundos: ", segundos

FinAlgoritmo
```

4. Diagrama de flujo



5. Prueba de escritorio

Nº	Proceso	c	días	resto	horas	minutos	segundos	Salida
1	Leer c	10000	0	0	0	0	0	Ingrese el número de segundos (c): 10000
2	días ← trunc(c/86400) resto ← c%86400	10000	0	10000	6	0	-	-
3	horas ← trunc(resto/3600) resto ← resto%3600	10000	0	2800	2	0	0	-
4	minutos ← trunc(resto/60) segundos ← resto%60	10000	0	2800	2	46	40	-
5	Escribir resultado	10000	0	2800	2	46	40	10000 equivale a 0 días, 2 horas, 46 minutos y 40 segundos.

Problema 1.6. Máximo, mínimo y media de alturas

Desarrolla un programa que lea por teclado un número indeterminado de alturas y deje de leer cuando se introduzca una negativa. El programa debe calcular e imprimir la media, el máximo y el mínimo de las alturas introducidas.

1. Análisis del problema

- Entrada: un número que representa una altura, se puede ingresar tantas veces hasta que ingrese un número negativo.
- Proceso: contar todos los datos ingresados, para, posteriormente obtener su media, el número máximo y mínimo.
- Salida: la media, la altura máxima y mínima
- Restricción: el programa deja de leer y procesar datos hasta que ingrese un número negativo.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	num	variable	Real
Dato de salida	media	variable	Real
Dato de salida	maximo	variable	Real
Dato de salida	minimo	variable	Real
Variable de proceso	suma	variable	Real
Variable de proceso	x	variable	Entero

3. Seudocódigo

```
Algoritmo AlturaMediaMaximoMinimo

    Definir num, suma, media, maximo, minimo Como Real
    Definir x Como Entero

    x ← 0
    suma ← 0
    Escribir "Engrese una altura (un valor negativo para salir): "
    Leer num

    Si num ≥ 0 Entonces
        maximo ← num
        minimo ← num
        Mientras num ≥ 0 Hacer
            suma ← suma + num
            x ← x + 1
            Si num > maximo Entonces
                maximo ← num
            Sino
                minimo ← num
            FinSi
```

```
    Escribir "Ingrese otra altura (dato negativo para salir)"
    Leer num
  FinMientras

  media ← suma/x

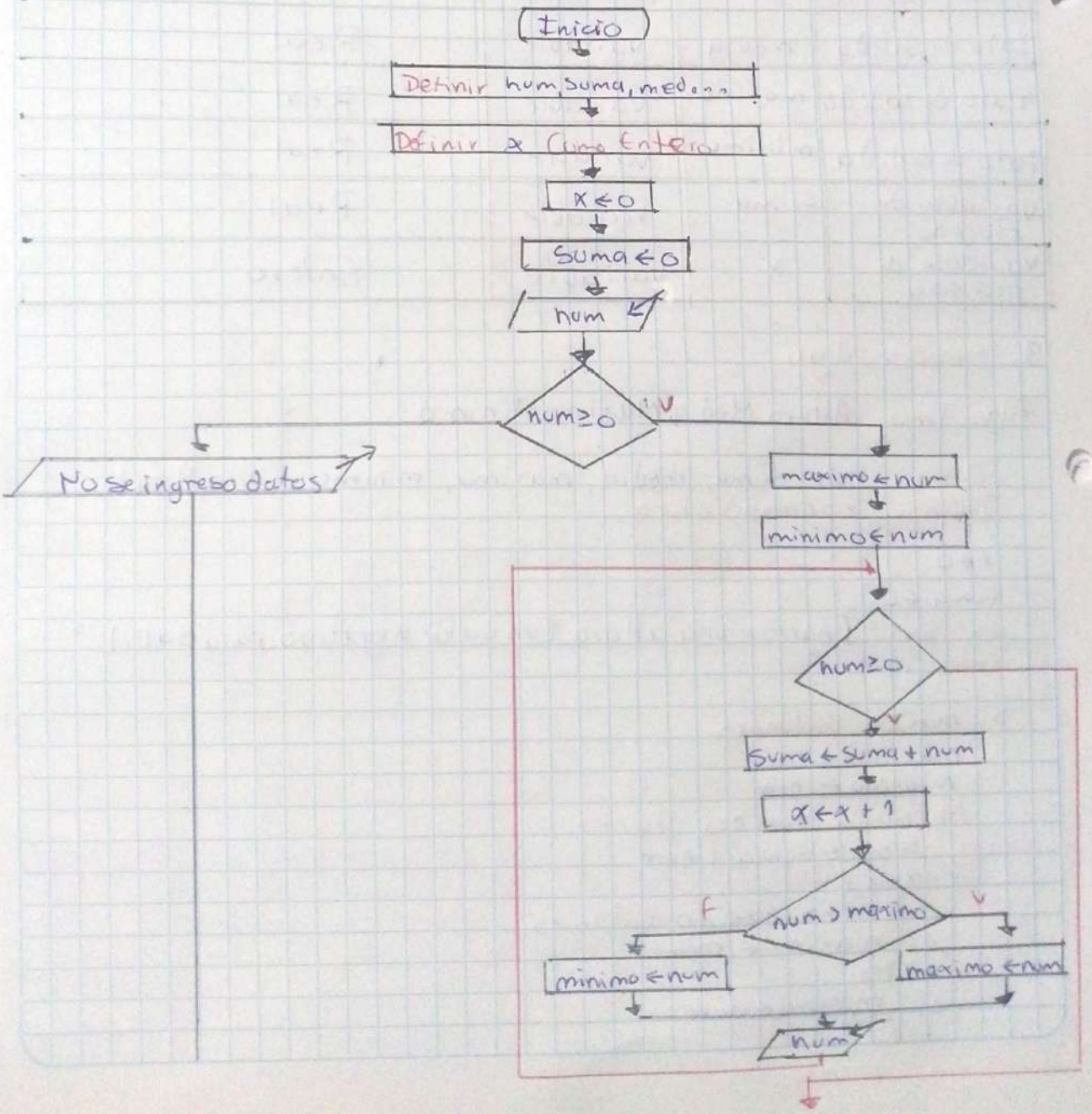
  Escribir "la media es: ", media
  Escribir "la altura máxima es: ", maximo
  Escribir "la atura minima es: ", minimo
SINO

  Escribir "No se ingresaron datos"

FinSi

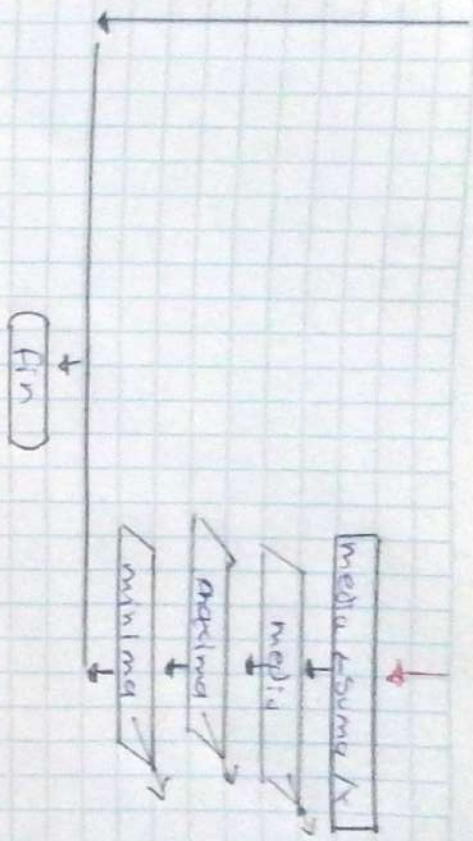
FinAlgoritmo
```

4. Diagrama de flujo:



Instrucciones	num	suma	x	maximo	minimo	media	Salida en Pantalla
leer num (primer valor: 170)	170	0	0	0	0	0	Ingrese una altura (negativo para terminar): 170
maximo ← num minimo ← num	170	0	0	170	170	0	-
1ª Vuelta: suma ← suma + num a ← a + 1	170	170	1	170	170	0	-
Leer num (segundo valor: 185)	185	170	1	170	170	0	Ingrese otra altura (negativo para terminar): 185
2° Vuelta: suma ← suma + num a ← a + 1 Actualiza maximo	185	355	2	185	170	0	-
Leer num (160)	160	355	2	185	170	0	Ingrese otra altura (negativo para terminar): 160
3ª Vuelta: Actualiza minimo	160	515	3	185	160	0	-
Leer num	-1	515	3	185	160	0	Ingrese otra altura (negativo para terminar): -1
Resultados:	-1	515	3	185	160	171,67	

5. Prueba de escritorio



Problema 17. Raiz cuadrada entera

Desarrolla un programa que calcule la raiz cuadrada entera por defecto de un número N positivo dado, que se define como el mayor número entero $R > 0$ tal que $R^2 \leq N$.

1. Análisis del problema

- Entrada: un número entero positivo
- Proceso: iniciar una variable i en 1 e incrementar su valor de uno en uno mediante un bucle mientras que el cuadrado de i sea mayor o igual a N ($i^2 \leq N$). Al finalizar, calcular la raiz cuadrada entera por defecto como $R = i - 1$.
- Salida: el valor de la raiz cuadrada entera por defecto (R) de N.
- Restricción: el valor ingresado para N debe ser un número positivo.

2. Tabla de objetos

Objetos	Nombre	Valor	Tipo
Dato de entrada	N	Variable	Entero
Dato de salida	R	variable	Entero
Variable de proceso	i	variable	Entero

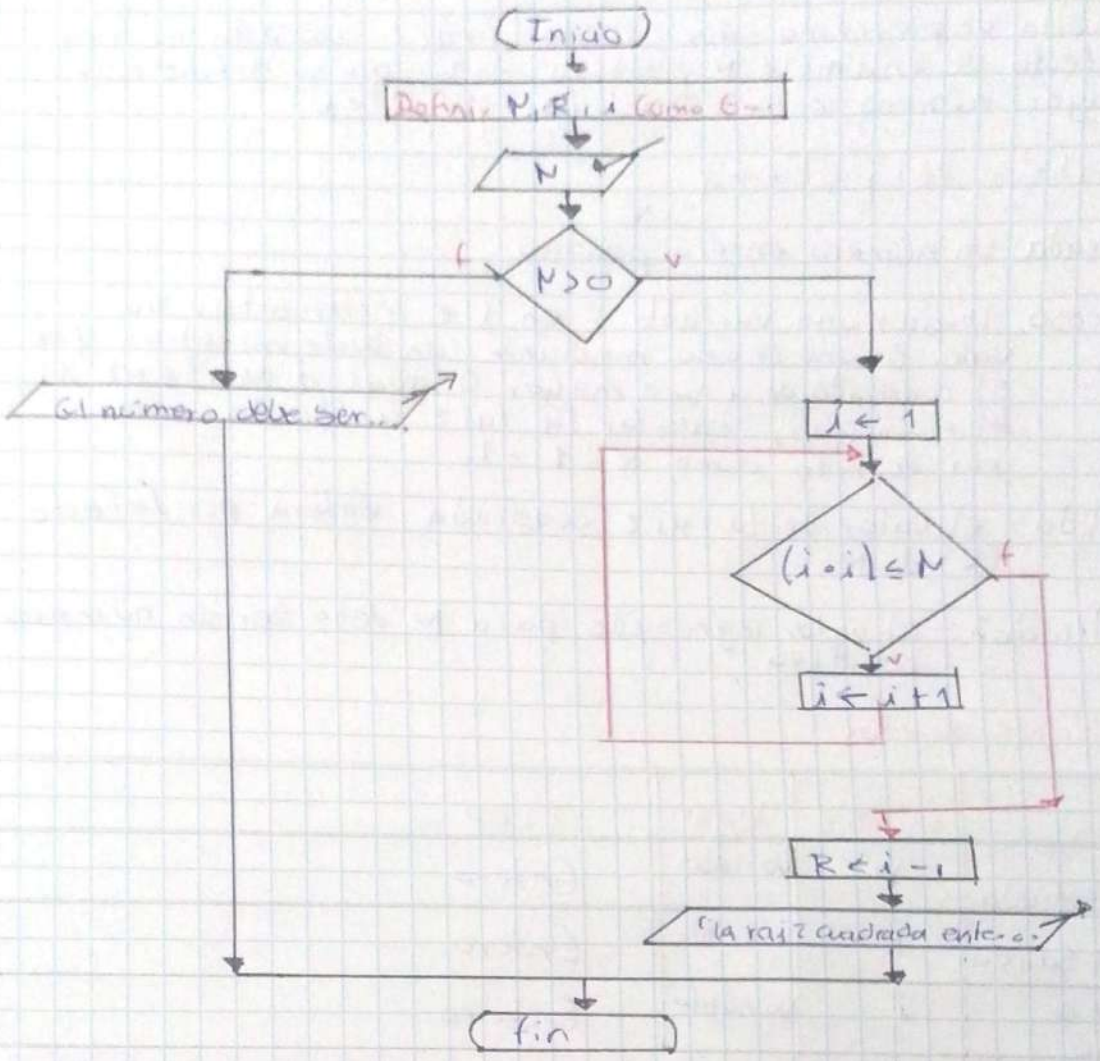
3. Seudocódigo

```
Algoritmo RaizCuadradaEntera
  Definir N, R, i Como Entero
  Escribir "Ingrese un número entero positivo (N): "
  Leer N

  Si N > 0 Entonces
    i ← 1
    Mientras (i * i) ≤ N Hacer
      i ← i + 1
    FinMientras

    R ← i - 1
    Escribir "La raiz cuadrada entera por defecto de", N, "es:", R
  Sino
    Escribir "El número debe ser mayor a 0."
  FinSi
Fin Algoritmo
```

4. Diagrama de Flujo



5. Prueba de escritorio

Operaciones	N	i	R	Salida en pantalla
leer N	10	—	—	Ingrese un número entero positivo (N): 10
(itera i = 1,2,3 hasta $i^2 > 10$)	10	4	—	—
R ← i - 1 (4-1)	10	4	3	—
Imprimir resultado	10	4	3	La raiz cuadrada entera por defecto de 10 es. 3

Problema 1.8 Número primo

Desarrolla un programa que ingrese por teclado un número entero mayor que 1 y determine si es primo o no.

1. Análisis del problema

Entrada: un número positivo mayor a 1.

Proceso: determinar si el valor ingresado (num>1), y luego a través de estructuras condicionales compuestas (num%2=0) si eso se cumple quiere decir que no es primo si no par, en caso contrario primo.

Salida: confirmar si es o no un número primo.

Restricción: el valor ingresado debe ser mayor a 1.

2. Tabla de objetos

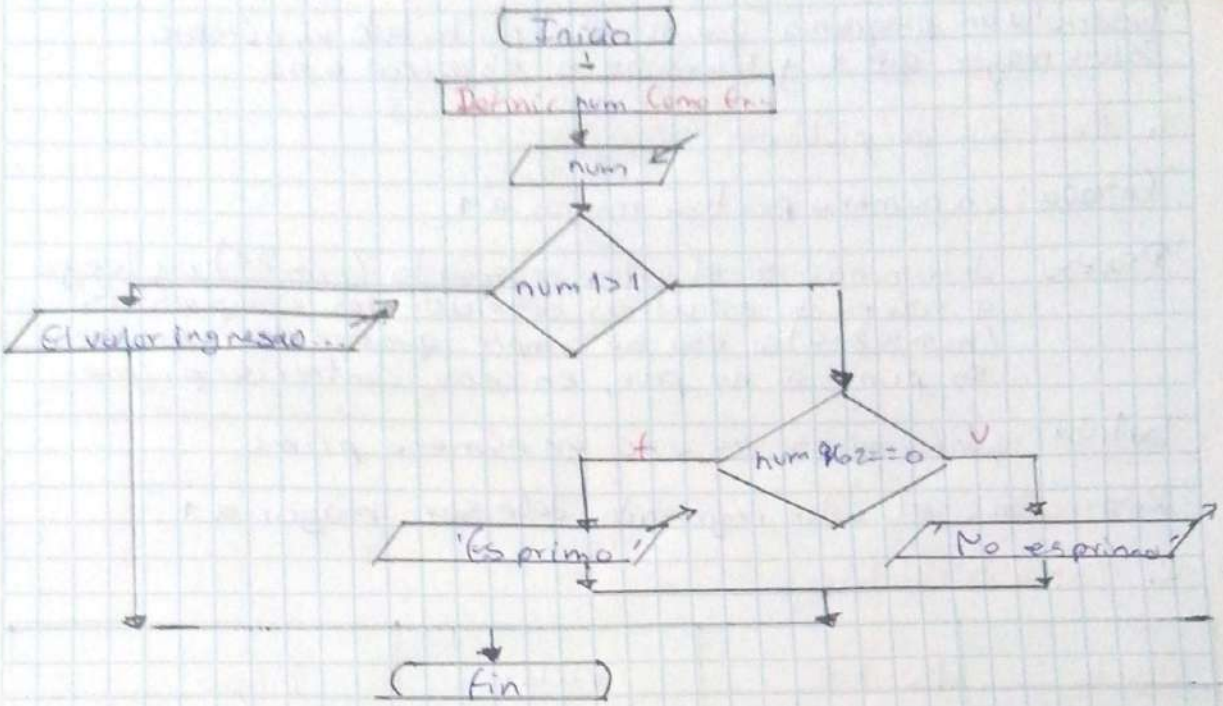
Objeto	Nombre	Valor	Tipo
Dato de entrada	num	variable	Entero
Dato de salida	Mensaje	variable	texto

3. Seudocódigo

```
Algoritmo NumeroPrimo
    Definir num Como Entero
    Escribir "Ingrese un número entero (mayor a 1):"
    Leer num

    Si num>1 Entonces
        Si num%2==0 Entonces
            Escribir "El valor ingresado no es un número primo"
        SiNo
            Escribir "El valor sin duda alguna es un número primo"
        FinSi
    SiNo
        Escribir "El valor ingresado no es mayor a 1"
    FinSi
FinAlgoritmo
```

4. Diagrama de flujo



5. Prueba de escritorio

Operación	Num	Salida en Pantalla
leer num	4	Ingrese un número entero (mayor que 1): 4
Evaluar num>1 (4>1, Verdadero)	4	-
num %2 ==0 (4%2==0, verdadero)	4	-
Ejecutar instrucción de la rama verdadera	4	El valor ingresado no es un numero primo

Problema 1.9 Número primos comprendidos entre dos números dados

Desarrolla un programa que muestre por pantalla todos los números primos comprendidos entre dos números enteros introducidos por teclado

1. Análisis del problema

- Entrada: dos números enteros (inicio y fin) para el intervalo de los números primos a mostrar.
- Proceso: Recorrer todos los numeros correspondientes en el intervalo. Para cada número, verificar si es primo mediante un bucle interno que cuente sus divisores exactos. Si el número es mayor que 1 y no tiene divisores exactos aparte de 1 y de sí mismo, se condisidera primo.
- Salida: Todos los números primos encontrados dentro del rango especificado, impresos en pantalla.
- Restricción: los valores ingresados deben ser números enteros.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	inicio	Variable	Entero
Dato de entrada	fin	Variable	Entero
Variable de control	i	variable	Entero
Variable de control	j	Variable	Entero
Variable de proceso	divisores	Variable	Entero
Variable auxiliar	temp	Variable	Entero
Dato de salida	Primos	Variable	Entero

3. Seudocódigo

```
Algoritmo PrimosRango
Definir inicio, fin, i, j, divisores, temp Como Entero
Escribir "Ingrese el número inicial:"
Leer inicio
Escribir "Ingrese el número final:"
Leer fin
```

```
Si inicio > fin Entonces
    temp ← inicio
    inicio ← fin
    fin ← temp
FinSi

Escribir "Numero primos entre", inicio, "y", fin, ":"

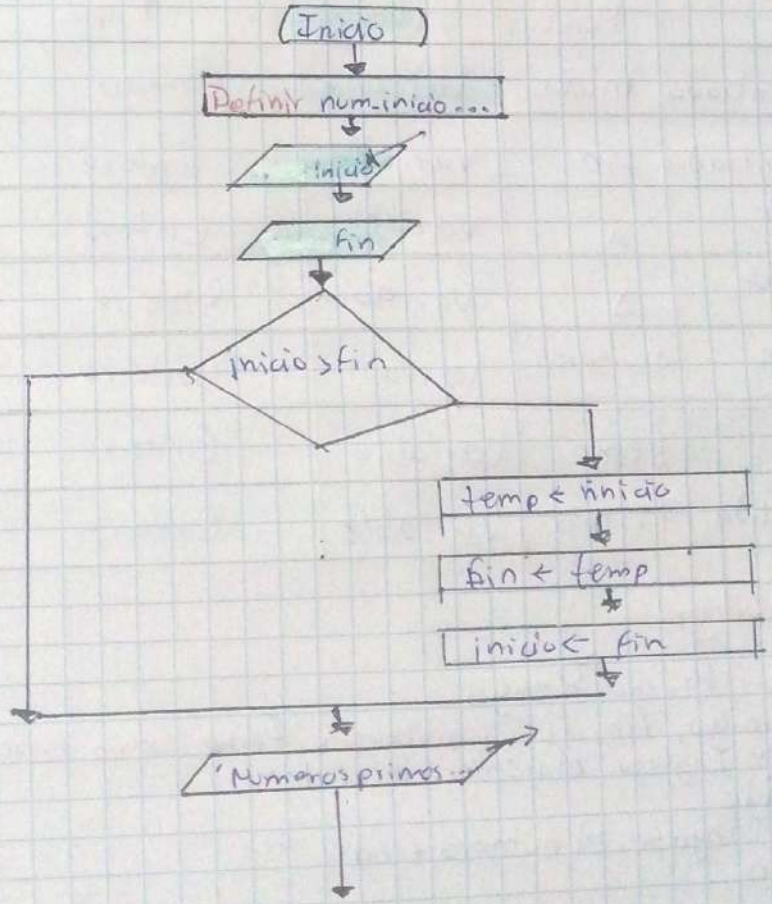
Para i ← inicio Hasta fin Hacer
    Si i > 1 Entonces
        divisores ← 0

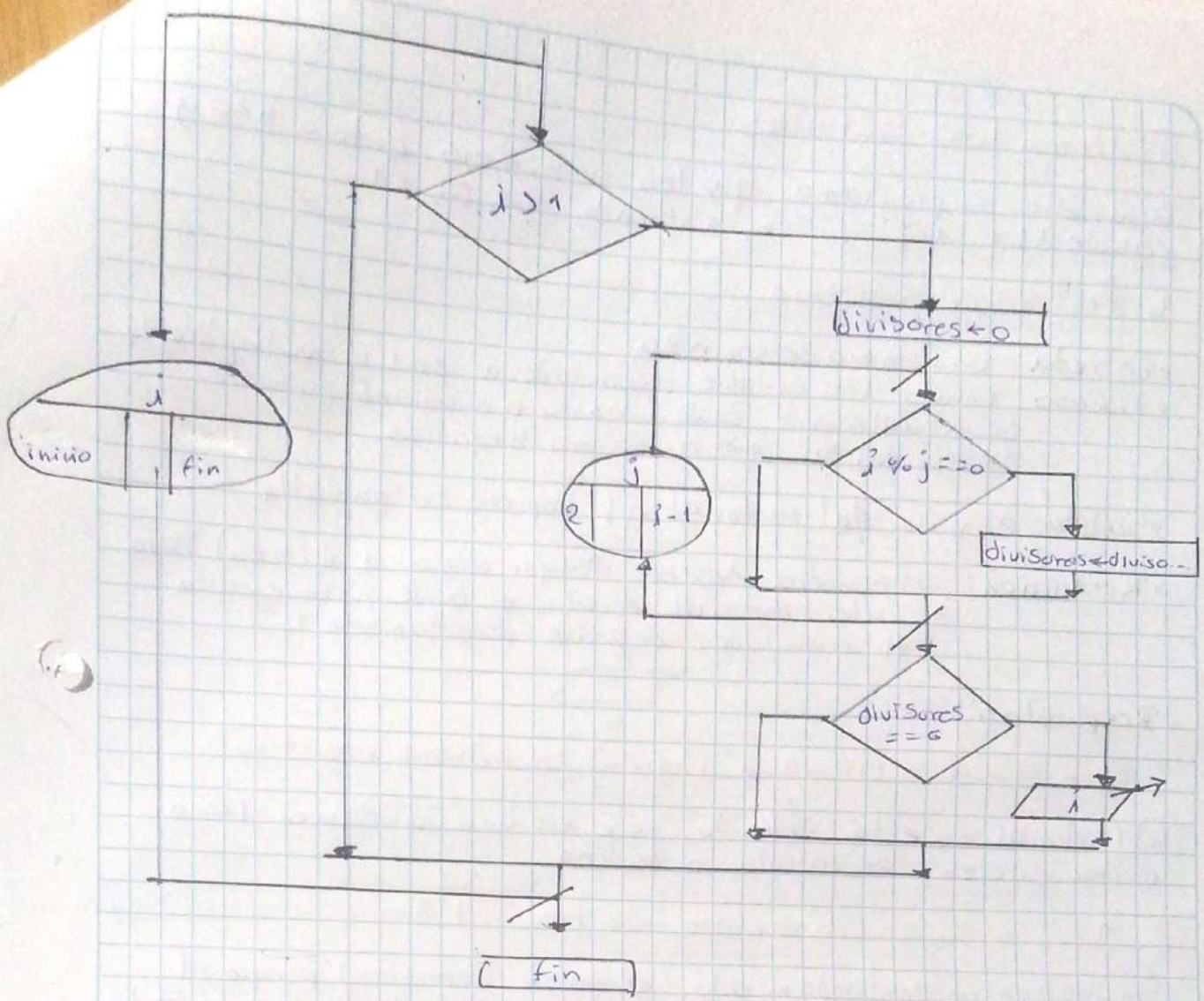
        Para j ← 2 Hasta i - 1 Hacer
            Si i % j == 0 Entonces
                divisores ← divisores + 1
            FinSi
        FinPara

        Si divisores == 0 Entonces
            Escribir i
        FinSi
    FinSi
FinPara

FinAlgoritmo
```

. U. Diagrama de flujo





5. Prueba de escritorio

Operación	Inicio	fin	i	divisores	Salida en Pantalla
Leer inicio leer. Fin	10	12	-	-	Ingrese el número inicial :10 Ingrese el número final: 12
Evaluar i = 10 (encuentra divisor, no es primo)	10	12	10	1	-
Evaluar i = 11 (sin divisores, es primo)	10	12	11	0	11
Evaluar i = 12 (encuentra divisor, no es primo)	10	12	12	4	-
Salida	10	12	12	4	11

Problema 1.10. Factorial

Desarrolla un programa que lea un número entero $n \geq 0$ y calcule $n! = 1 \cdot 2 \cdot \ldots \cdot n$, sabiendo que $0! = 1$.

1. Análisis del problema

- Entrada: un número entero $n \geq 0$
- Proceso: Iniciar una variable acumuladora en 1 y multiplicarla secuencialmente desde 1 hasta n mediante un bucle. Se debe validar que n no sea negativo.
- Salida: el valor del factorial n! impreso en pantalla.
- Restricción: el número debe ser mayor o igual a 0 ($n \geq 0$). Para el caso especial de 0!, el bucle no se ejecuta y el acumulador devuelve directamente 1.

- Respuestas:

¿Qué ocurre si el usuario ingresa un número negativo?

El factorial no está definido; requiere una validación para evitar errores de cálculo o bucles.

¿Y con n=20 en una variable de tipo int?

Se produce un desbordamiento de entero (overflow) porque el resultado ($\approx 2.43 \times 10^{18}$) supera la capacidad de almacenamiento estándar, por lo que se debe usar el tipo Real.

2. Tabla de Objetos

Objeto	Nombre	Valor	Tipo
Datos de entrada	n	Variable	Entero
Variable de control	i	Variable	Entero
Datos de salida	factorial	Variable	Real

3. Seudocódigo

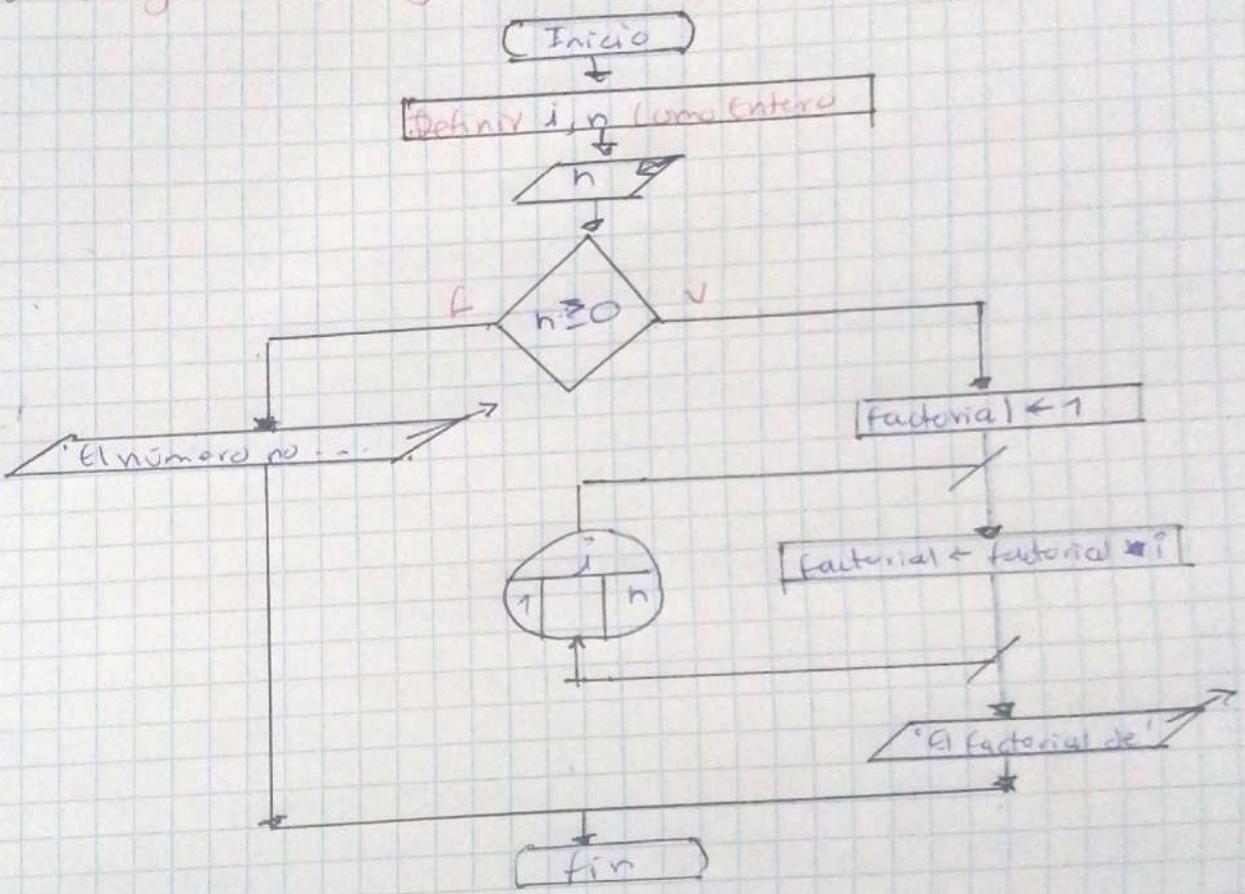
```
Algoritmo CalculoFactorial
  Definir n, i Como Entero
  Definir factorial Como Real

  Escribir "Ingrese un número entero n>=0: "
  Leer n

  Si n>=0 Entonces
    factorial ← 1
```

```
    Para i ← 1 Hasta n Hacer
        factorial ← factorial * i
    FinPara
    Escribir "El factorial de ", n, " es: ", factorial
SiNo
    Escribir "El número no puede ser negativo"
FinSi
FinAlgoritmo
```

4. Diagrama de flujo



5. Prueba de escritorio

Operación	n	i	factorial	Salida en Pantalla
Leer n	4	-	-	Ingrese un número entero n>=0: 4
Validar n>=0 (verdadero) y fijar factorial ← 1	4	-	1	-

Bucle Para i ← 1 Hasta 4 i=(1*1)	4	1	1	-
i = 2 (1*2)	4	2	2	-
i = 3 (2*3)	4	3	6	-
i = 4 (6*4)	4	4	24	-
Salida	4	4	24	El factorial de 4 es: 24